

Example Quarto File

Annamalai Vikash Raam Manoj

Introduction

This document demonstrates GitHub collaboration and version control using Git branches.

Step 1: Creating a Quarto Document

A new project was created in Positron to organise the workflow. Inside this project, a Quarto document named `example.qmd` was created. The file contains a simple R code chunk to demonstrate execution within Quarto.

Example Code

The following code shows a preview of the `mtcars` dataset and calculates the average miles per gallon.

```
head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

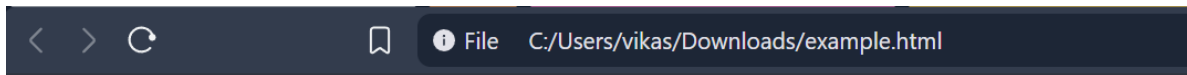
```
mean(mtcars$mpg)
```

```
[1] 20.09062
```

The document was rendered into HTML using the following command in the terminal:

```
quarto render example.qmd --to html
```

This generated an HTML file, confirming that the Quarto document renders correctly and executes R code.



Example Quarto File

Author

Annamalai Vikash Raam Manoj

Introduction

This document demonstrates a simple Quarto file that can be rendered into HTML.

Example Code

The following code shows a preview of the `mtcars` dataset and calculates the average miles per gallon.

```
head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
mean(mtcars$mpg)
```

```
[1] 20.09062
```

Figure 1: Output of sample code in the example html file

Step 2: Initialising Git and Pushing to GitHub

- The project folder was initialised as a Git repository using the command: `git init`
This creates a hidden `.git` directory that allows Git to track changes in the project.

- All files in the project were added to the staging area using:

```
git add .
```

This prepares the files to be committed.

- An initial commit was created using:

```
git commit -m "Initial commit: added example quarto file and rendered  
HTML output"
```

This saves the current state of the project in the Git history.

- A remote repository was then created on GitHub and linked to the local repository using:

```
git remote add origin <repository-url>
```

```
git branch -M main
```

```
git push -u origin main
```

These commands connect the local repository to GitHub and push the committed files to the remote repository, making them available online.

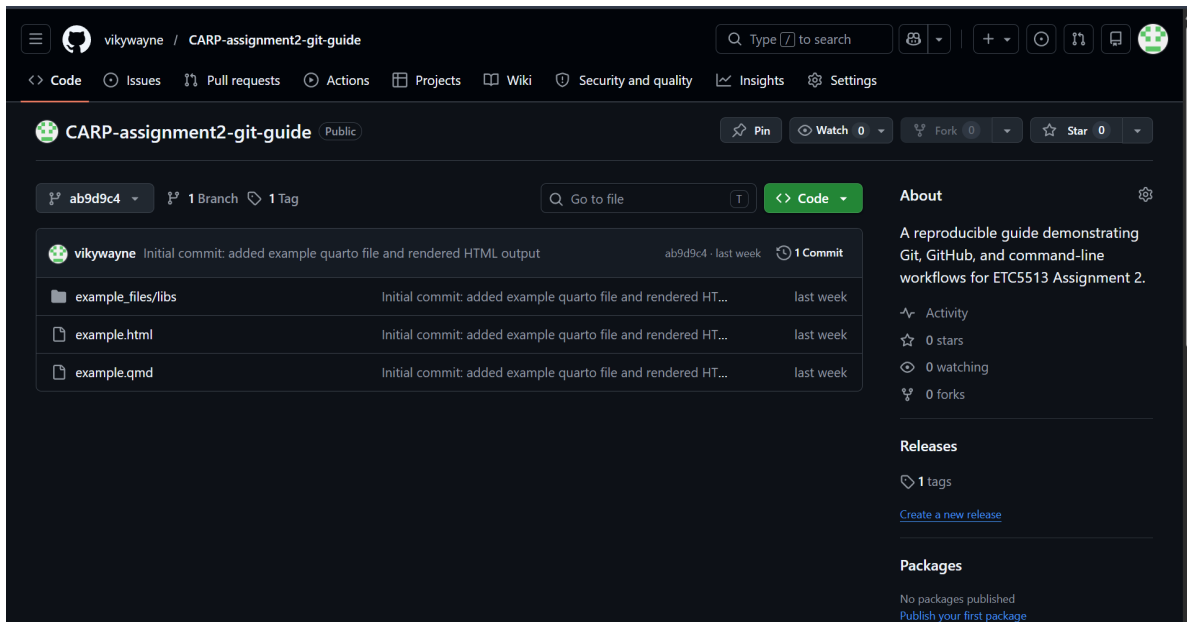


Figure 2: snapshot of Github repository with all the files from local repository

Step 3: Creating and Working on a Branch

- A new branch named `testbranch` was created and checked out using:

```
git checkout -b testbranch
```

This creates a separate branch for development without affecting the `main` branch.

- The `example.qmd` file was modified in the branch and the changes were staged.
- The changes were committed using:

```
git commit -m "Updated introduction in testbranch"
```

This saves the modifications in the branch history.

- The branch was then pushed to GitHub using:

```
git push -u origin testbranch
```

This creates a remote version of the branch and links it to the local branch.

Step 4: Amending a Previous Commit

- A `data` folder was created and the dataset from Assignment 1 was added to it.
- Instead of creating a new commit, the previous commit was amended using:

```
git commit --amend -m "Updated introduction and added data folder"
```

- This updates the most recent commit by including the newly added files.

The amended commit was pushed to GitHub using:

```
git push --force
```

Force push was required because amending a commit changes the commit history and creates a new commit hash.

Step 5: Creating a Merge Conflict on Main

- The `main` branch was checked out again and the introduction section in `example.qmd` was modified differently from the version in `testbranch`.
- The updated file was committed using:

```
git commit -m "Modified introduction in main branch for merge conflict"
```

- The changes were then pushed to GitHub.

Modifying the same section differently in both branches intentionally created a merge conflict scenario.

Step 6: Merging and Resolving a Conflict

- The changes from `testbranch` were merged into the `main` branch using:

```
git merge testbranch
```

- Git detected a merge conflict because the same line had been modified differently in both branches.
- The conflict markers were manually removed and the correct final version of the text was retained.
- After resolving the conflict, the merge was completed using:

```
git commit -m "Resolved merge conflict between main and testbranch"
```

- The resolved merge was then pushed to GitHub using:

```
git push
```

This updated the remote repository with the merged changes.

Step 7: Creating an Annotated Tag

- An annotated tag named `v1.0` was created using:

```
git tag -a v1.0 -m "Version 1.0 after merge conflict resolution"
```

This tag marks a stable project version after resolving the merge conflict.

- The tag was pushed to GitHub using:

```
git push origin v1.0
```

This made the version tag available in the remote repository.

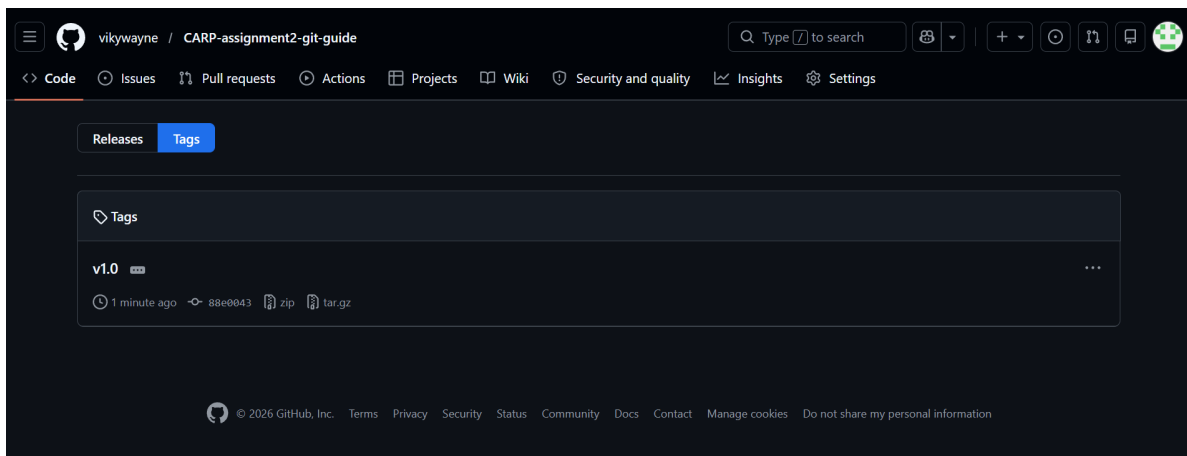


Figure 3: tag shown in Github repository

Step 8: Deleting the Branch

- After the merge was completed, the `testbranch` branch was deleted locally using:

```
git branch -d testbranch
```

- The remote version of the branch was deleted from GitHub using:

```
git push origin --delete testbranch
```

Removing unused branches helps keep the repository organised.

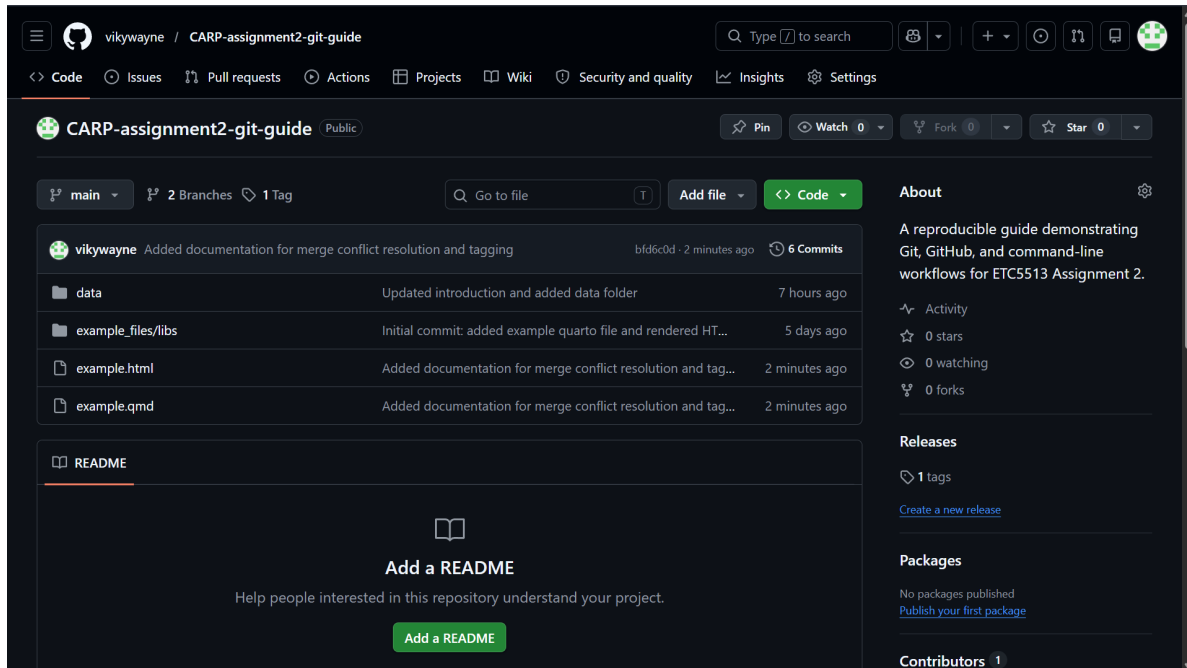


Figure 4: github repo page before deleting the branch in remote

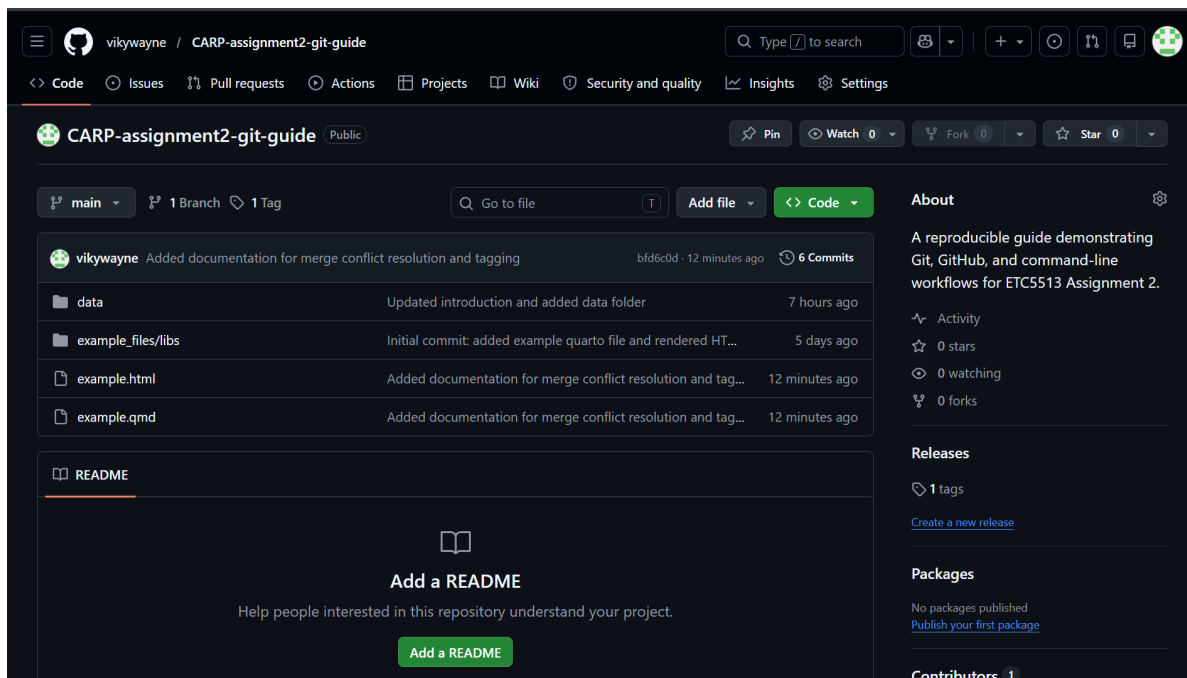


Figure 5: github repo page after deleting the branch in remote

Step 9: Viewing the Commit History

- The condensed commit history was displayed using:

```
git log --oneline
```

This command provides a simplified view of the commit history, showing commit hashes and commit messages.

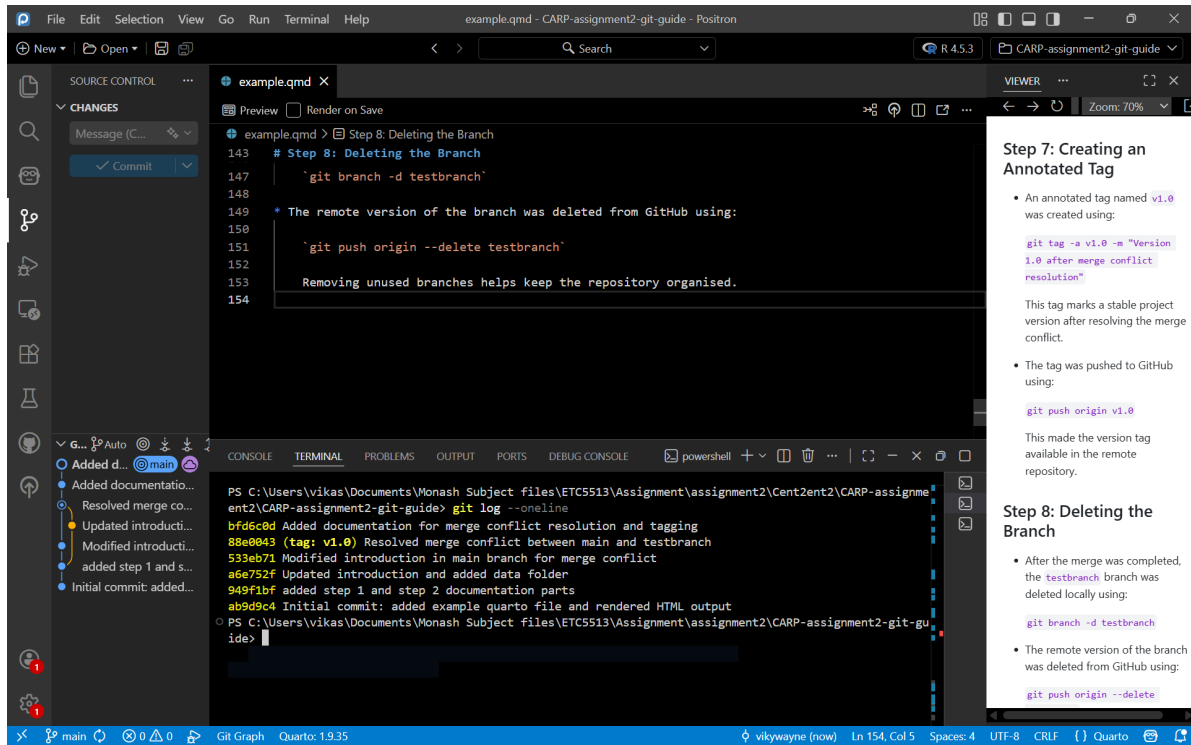
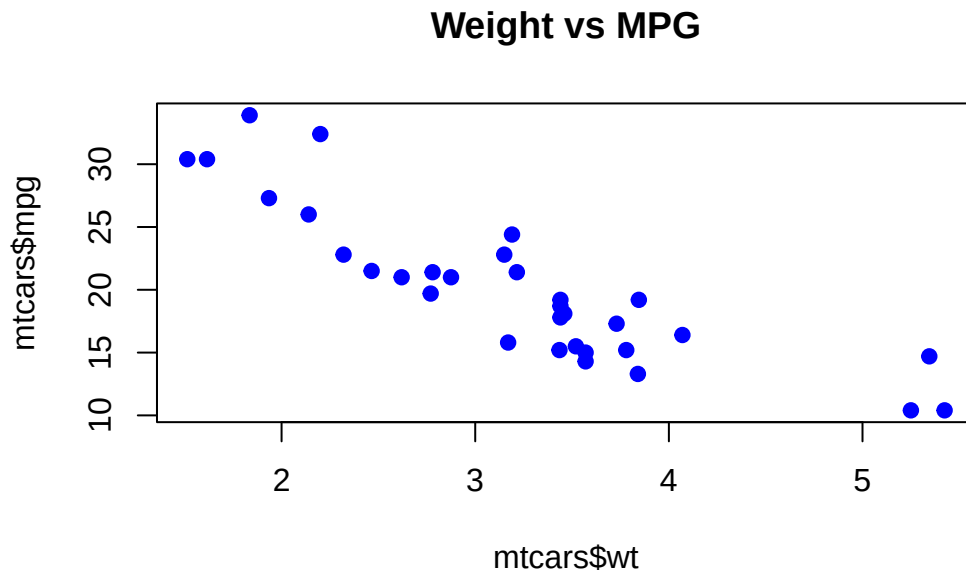


Figure 6: Condensed git commit history

Step 10: Undoing a Commit Without Losing Changes

- A new section containing a simple plot was added to the Quarto document.



- The changes were committed using:

```
git commit -m "Added plot section for reset demonstration"
```
- The commit was then undone without removing the local file changes using:

```
git reset --soft HEAD~1
```

This removed the most recent commit from the Git history while keeping the changes available locally.

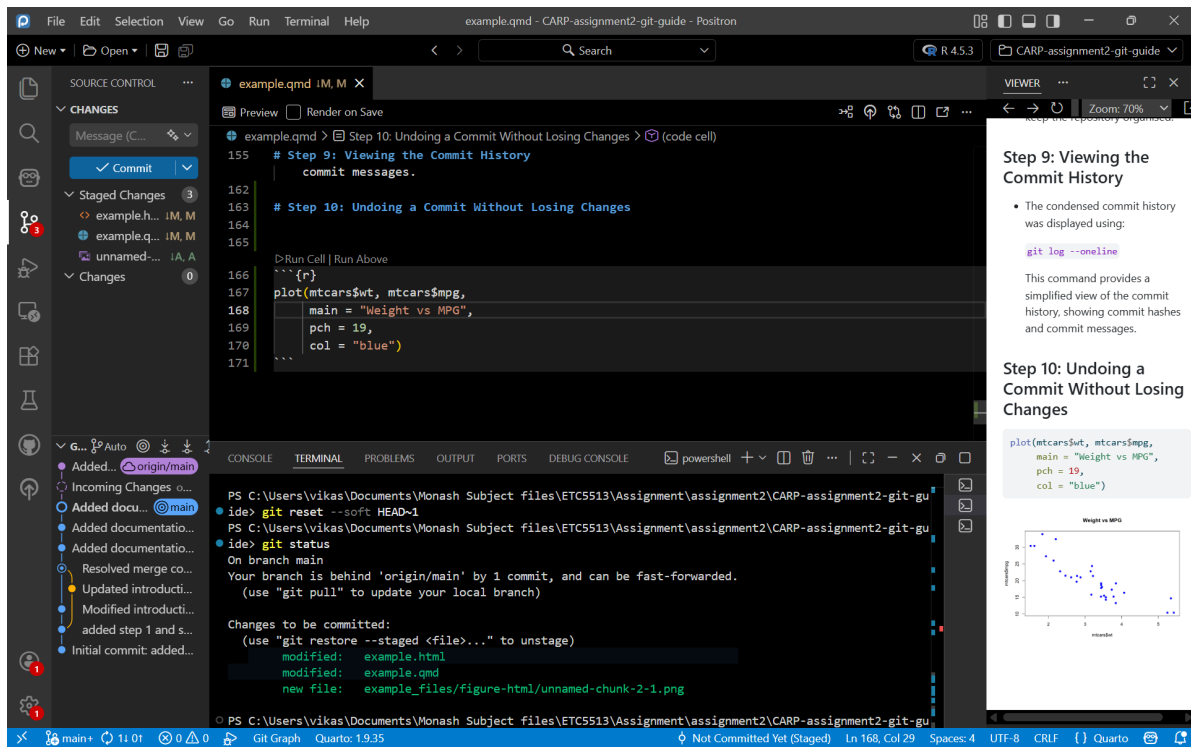


Figure 7: terminal showcasing git soft reset in process